MOTEUR DE RECHERCHE



Chushpan, mon chat

Introduction

Le cadre est le projet du cours de Programmation de spécialité du master 1 Informatique en alternance à l'Université Lumière Lyon 2. L'idée de ce projet est de construction progressive un moteur de recherche sure une base des documents liés par thème. L'objectif est de comprendre les différentes étapes pour passer d'une collection de textes à un outil capable de retrouver et classer les documents à partir d'une requête utilisateur via une interface.

Le theme initialement choisi était sur les chats. Les données Reddit ont été collectées dans le subreddit consacré aux chats mais la récupération depuis arXiv a dû être élargie aux animaux pour obtenir un corpus exploitable.

Le travail était organisé en trois étapes. La collecte et la structuration des documents, l'analyse et le classement des résultats et enfin la création d'interfaces et de fonctionnalités complémentaires.

La representation des documents, la pondération des termes et le calcul des scores ont été codés dans le projet. Plusieurs bibliothèques sont quand même utilisées pour la manipulation des tableaux, le calcul numérique, les interfaces et les visualisations.

Le programme reste un prototype de recherche dans un corpus local. Il n'explore pas l'ensemble du Web et ne cherche pas à reproduire toutes les fonctionnalités d'un moteur de recherche généraliste comme Google ou autre.

Les spécifications

Besoin et objectifs

Le besoin principal est de permettre à une personne de retrouver des informations dans une collection de textes dans devoir écrire du code Python. L'utilisateur doit pouvoir saisir mot-clés, choisir un nombre maximal de résultats et consulter les documents proposés par le moteur. Le programme doit aussi permettre d'explorer le corpus autrement que par le classement des résultats.Un concordancier, des statistiques lexicales et des graphiques élargissent la recherche sur l'ensemble des documents.

Les principales fonctionnalités attendues sont suivantes:

- **Collecte des données**: récupérer des textes sur Reddit et arXiv avec des métadonnées.
- **Gestion du corpus**: organiser les documents et les auteurs, ensuite sauvegarder et recharger les données.
- **Analyse textuelle**: nettoyer les textes, construire un vocabulaire et calculer les fréquences des mots.
- **Recherche documentaire**: représenter les documents et la requête, calculer leurs similarités et classer les résultats.
- **Interface utilisateur**: proposer un champ de saisie, un curseur de limitation et une zone de résultats.
- **Consultation des documents**: afficher le texte sauvegardé et permettre l'ouverture de la page d'origine.

Contraintes de réalisation

Le programme doit conserver une séparation entre les classes centrales et les interfaces. Le classement des résultats ne doit pas être réimplémenté en plus dans chaque interface (Jupiter et Streamlit). Les deux parcours utilisent la classe `SearchEngine`.
Le lancement normal doit pouvoir fonctionner à partir du corpus sauvegardé. Il n'est pas nécessaire de réinterroger Reddit ou arXiv à chaque utilisation. Cela rend les essais plus reproductibles et évite de dépendre des API pour consulter les données déjà collectées.
Les dépendances doivent être recensées dans `requirements.txt`, les fichiers doivent être organisés de manière compréhensible et les identifiants de connexion ne doivent pas être publiés.
Les chemins utilisés doivent être relatifs au projet.
La version du Python préférable est 3.13 et les bibliothèques importées doivent être utilisées.

Extensions réalisées

Plusieurs fonctionnalités ont été ajoutées au moteur principal:

- **Filtre par source**: sélectionner les documents Reddit, arXiv ou l'ensemble du corpus.
- **Application Streamlit**: proposer une interface web locale en complément du notebook.
- **Concordancier interactif**: afficher les occurrences d'un mot avec leurs contextes.
- **Statistiques lexicales**: représenter les mots fréquents du corpus après exclusion de certains mots vides.
- **Comparaison des sources**: faire apparaître des mots communs et des mots spécifiques aux deux sous-corpus.
- **Évolution temporelle**: suivre les occurrences d'un mot par mois de publication.
- **Lecteur intégré**: consulter le texte stocké d'un résultat et ouvrir sa page d'origine.

Ces extensions complètent la recherche. Elles rendent le corpus plus facile à consulter et permettent d'observer ses caractéristiques.

L'analyse

Environnement de travail

Le projet a été développé en Python avec Visual Studio Code. Les notebooks Jupyter ont aidé à exécuter progressivement les traitements, inspecter les résultats et construire la première interface.

Les bibliothèques utilisées:

- **pandas**: manipulation des tableaux, lecture et écriture des fichiers, présentation des résultats.
- **NumPy**: opérations numériques sur les vecteurs.
- **SciPy**: représentation des matrices.
- **praw**: accès aux données Reddit.
- **urllib et xmltodict**: récupération et interprétation des données arXiv.
- **ipywidgets**: création des contrôles interactifs dans le notebook.
- **Streamlit**: création de l'application web locale.
- **Matplotlib**: réalisation des graphiques.
- **tqdm**: affichage de la progression de la construction de la matrice.
- **unittest**: automatisation des tests ciblés.

Git a été utilisé pour conserver les étapes du développement. Les versions `v1`, `v2` et `v3` correspondent aux grandes étapes du projet.

Constitution et caractéristiques du corpus

Le corpus rassemble deux types de textes. Reddit fournit des publications d'un espace de discussion consacré aux chats et arXiv fournit des résumés de travaux scientifiques obtenus avec une requête plus large autour des animaux.
Ce choix permet de visualiser et comparer des contenus différentes. Les publications Reddit et les résumés scientifiques ne sont pas écrit avec le même objectif et présentent des différences de vocabulaire, de longueur et de formulation.
Chaque document conserve des informations communes: titre, auteur principal, date, URL, texte et source. Et les informations spécifiques à chaque source sont aussi présentes, comme le nombre de commentaires pour Reddit ou les coauteurs pour arXiv.
Lors de la collecte, les textes de moins de 100 caractères sont écartés pour conserver des contenus suffisamment développés pour être analysés.

Deux fichiers conservent des données dans le projet:

- **`corpus.csv`**: tableau des textes et de leurs sources.
- **`corpus_objets.csv`**: documents accompagnés de leurs métadonnées, utilisé pour reconstituer le corpus.

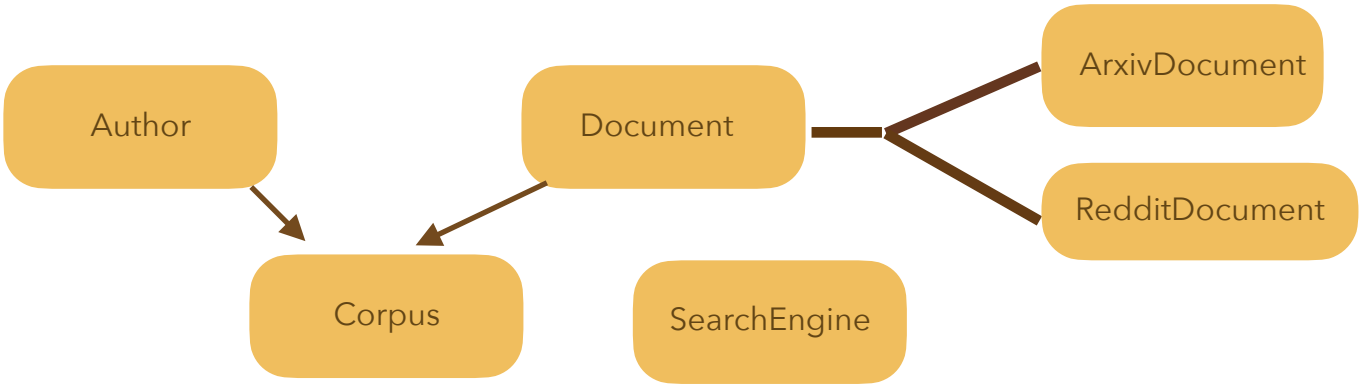
Avec extension `.csv` ces fichiers utilisent une tabulation comme séparateur.
Le corpus final contient 154 documents, dont 54 documents Reddit et 100 documents arXiv. Après nettoyage, le vocabulaire comporte 4657 termes.

Modélisation des données

L'organisation du programme repose sur plusieurs classes avec des responsabilités personnifiés:

- **`Document`**: représente les informations d'un document.
- **`RedditDocument` et `ArxivDocument`**: spécialisent les documents selon leur source.
- **`Author`**: regroupe les documents associés à un auteur.
- **`Corpus`**: gère les documents, les auteurs, la sauvegarde, le chargement et certaines analyses.
- **`SearchEngine`**: construit les représentations numériques et classe les documents.
- **`DocumentFactory`**: centralise la création des documents en fonction de leur type.

L'héritage permet de conserver une structure commune tout en représentant les particularités des sources - nombre de commentaires Reddit par exemple. La méthode `getType()` permet aux traitements de connaître la source d'un document sans devoir modifier leur fonctionnement pour chaque classe fille.



La conception

Organisation du travail

Le projet a été réalisé individuellement. Le développement a suivi la progression proposée dans les trois parties du sujet, puis une phase de perfection a aidé de corriger certains comportements maladroits et d'améliorer le programme.
La première version a posé les bases comme collecte, classes, auteurs et sauvegarde. La deuxième a ajouté les analyses textuelles et le moteur de recherche. La troisième a rendu ces fonctionnalités accessibles par une interface et a intégré les extensions.
La phase de perfection ne concernait pas uniquement des erreurs provoquant un arrêt total de l'execution mais aussi a fixé des comportements fonctionnellement maladroits comme l'affichage de documents à score nul ou quand le filtre ne fonctionnant pas sur concordancier.

Nettoyage des textes

Avant la construction du vocabulaire les textes sont normalisés par la méthode `nettoyer_texte`. Les caractères sont convertis en minuscules, les sauts de ligne et la ponctuation sont remplacés par des espaces, les chiffres sont supprimés et les espaces successifs sont regroupés.
Cette étape permet de rapprocher des formes qui ne diffèrent que par la casse ou certains caractères de présentation. La requête de l'utilisateur est nettoyée avec la même méthode pour être mieux comparable aux documents.
Ce nettoyage malheureusement entraîne une perte d'information. En plus la version actuelle ne rapproche pas automatiquement des synonymes ou toutes les variantes d'un mot.
L'exclusion des mots vides dans les graphiques constitue un traitement distinct. Retirer `with` ou `that` d'une visualisation des statistiques n'a pas supprimé ces termes du vocabulaire utilisé par le moteur.

Construction du vocabulaire et de la matrice TF

Le vocabulaire associe chaque mot distinct à un identifiant. Cet identifiant correspond à une colonne de la matrice et chaque ligne représente un document.
La matrice TF contient le nombre d'occurrences de chaque terme dans chaque document. Par exemple, un document contenant deux fois `cat` possède la valeur 2 dans la colonne correspondante.

La matrice comporte de nombreuses valeurs nulles car tous les documents ne contiennent qu'une partie du vocabulaire. Une représentation creuse est utilisée pour stocker principalement les valeurs non nulles.
La construction des comptes s'effectue en parcourant les mots de chaque document. Une barre de progression permet de visualiser l'avancement de cette étape.

Pondération TF-IDF

Les fréquences brutes ne suffisent pas à distinguer les termes très répandus des termes plus discriminants. Le moteur utilise une pondération TF-IDF.
Dans l'implémentation retenue l'IDF d'un terme *t* est calculé par:

$$\mathrm{idf}(t) = \ln\left(\frac{N}{\mathrm{df}(t)}\right)$$

où *N* représente le nombre total de documents et *df(t)* le nombre de documents contenant le terme.
Le poids du terme dans un document *d* est ensuite:

$$w_{d,t} = \mathrm{tf}(d,t) \times \mathrm{idf}(t)$$

La fréquence TF correspond ici au nombre brut d'occurrences. Un terme présent dans tous les documents possède un IDF nul avec cette formule et ne contribue pas au score.
La requête est représentée dans le même vocabulaire, avec les mêmes coefficients IDF. Les mots inconnus sont ignorés et, si aucune composante exploitable ne subsiste, le moteur retourne un tableau vide.

Similarité cosinus et sélection des résultats

Le classement repose sur la similarité cosinus entre le vecteur d'un document et celui de la requête:

$$\mathrm{score}(d,q) = \frac{\mathbf{w}_d \cdot \mathbf{w}_q}{\|\mathbf{w}_d\| \times \|\mathbf{w}_q\|}$$

Le score dépend à la fois des termes communs et de la norme des vecteurs. Il ne correspond ni à un simple nombre d'occurrences ni à un pourcentage de pertinence.
Cela explique qu'un mot présent deux fois dans un document puisse produire un score voisin de 0.1. Les autres termes du document interviennent dans la représentation et sa normalisation.

Après le calcul, le moteur conserve uniquement les scores strictement positifs et les documents appartenant à la source sélectionnée. Les résultats sont triés par score décroissant et le nombre de résultats affichées est limité.
L'ordre de ces opérations est important. Si la source était filtrée seulement après la sélection des premiers résultats globaux, certains documents valides de la source demandée pourraient être éliminés trop tôt.

Concordancier et lecture des documents

Le concordancier répond à un besoin différent du moteur de recherche. Il repère les occurrences d'un mot et affiche leur contexte gauche et droit.
La recherche est réalisée dans chaque document séparément, car sinon un contexte situé à la fin d'un texte serait prolongé par le début du document suivant.
Le repérage est insensible à la casse et utilise des frontières de mots. Comme ça, dans les données de test, `CAT` est reconnu comme une occurrence de `cat`, contrairement à `bobcat`.
Le nombre de documents et le nombre d'occurrences peuvent donc différer. Deux documents trouvés par le moteur peuvent contenir trois occurrences si le mot apparaît deux fois dans l'un d'eux.

Interfaces et analyses complémentaires

L'interface notebook utilise des objets ipywidgets pour la saisie, les curseurs, les boutons et les zones de sortie. Chaque événement lit les paramètres, appelle le traitement correspondant et remplace le contenu affiché.
L'application Streamlit utilise le même moteur de recherche. Elle organise les fonctionnalités en onglets et propose des panneaux dépliables pour consulter les documents.
Les boutons Statistiques et Comparer ont été séparés des actions liées à la requête dans le notebook. Cette disposition évite de laisser penser que ces analyses globales dépendent des saisis de l'utilisateur.
La comparaison traite chaque source comme un gros document. Avec l'IDF employé sur deux sous-corpus, les mots présents dans les deux ont un poids nul. Les graphiques présentent des mots absents de l'autre source après nettoyage et filtrage.
L'évolution temporelle regroupe les occurrences du premier mot nettoyé par mois de publication. Les dates inutilisables sont ignorées et les mois intermédiaires sans occurrence sont représentés par zéro.
La courbe montre des occurrences brutes dans les données disponibles. Elle ne constitue pas une mesure générale de popularité, car une augmentation peut aussi provenir d'un plus grand nombre de documents collectés sur une période. Les documents de source Reddit sont majoritairement récupérés d'un seul mois.

Les problèmes rencontrés

Accès aux données et environnement d'exécution

Une première difficulté a concerné l'obtention des identifiants pour la collecte depuis Reddit. La solution a été d'utiliser les accès fournis par l'enseignant.
L'utilisation de Jupyter a été une nouveauté personnelle et a demandé un apprentissage. Les difficultés concernaient l'installation du noyau, le choix de l'interpréteur et les différences entre les environnements Python.
Une erreur récente lors du lancement des tests sur une autre machine montre ce problème: l'interpréteur utilisé ne disposait pas de `pandas` et cela empêchait l'import du corpus.

Cohérence des résultats et des filtres

L'ajout du filtre source a provoqué une erreur `KeyError: source`. L'interface attendait cette colonne dans les résultats alors que l'objet utilisé ne la fournissait pas encore.
La correction était de relancer le noyau complètement pour effacer les variables locaux datées et récupérer le nouveau SearchEngine avec la source implémentée dans le retour.
Un autre problème concernait les résultats à score nul. Quand la limite était élevée, des documents sans score positif apparaissaient pour compléter le tableau.
La correction a été de retirer ces documents avant le classement final et la limitation.

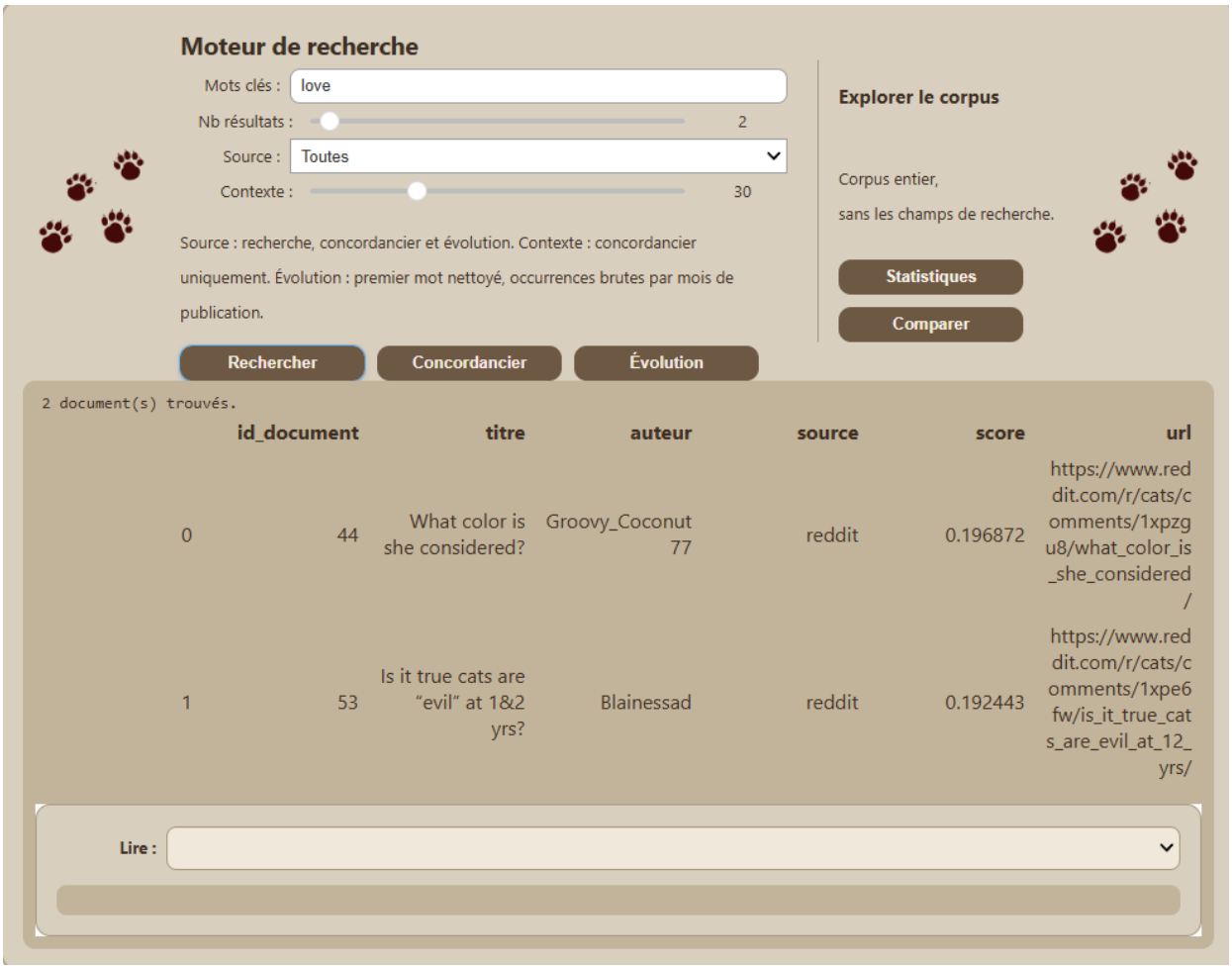
Ergonomie et consultation

Les premiers résultats indiquaient le titre, l'auteur et le score mais ne permettaient pas de lire le texte. Le moteur retrouvait des documents sans offrir une possibilité de consultation.
L'ajout du lecteur et du lien d'origine a enlevé cette limite.

Exemple commenté d'utilisation

L'exemple suivant présente le parcours dans le notebook. Il suppose que les dépendances sont installées et que le fichier `corpus_objets.csv` est présent dans le projet.
L'utilisateur ouvre `interface.ipynb`, sélectionne le bon environnement Python et exécute les cellules dans l'ordre. Le corpus est chargé, le moteur est construit et l'interface apparaît.
Il saisit ensuite une requête, par exemple `love`, puis règle le nombre maximal de résultats. Le filtre permet de conserver l'ensemble des sources ou de limiter la recherche à Reddit ou arXiv.

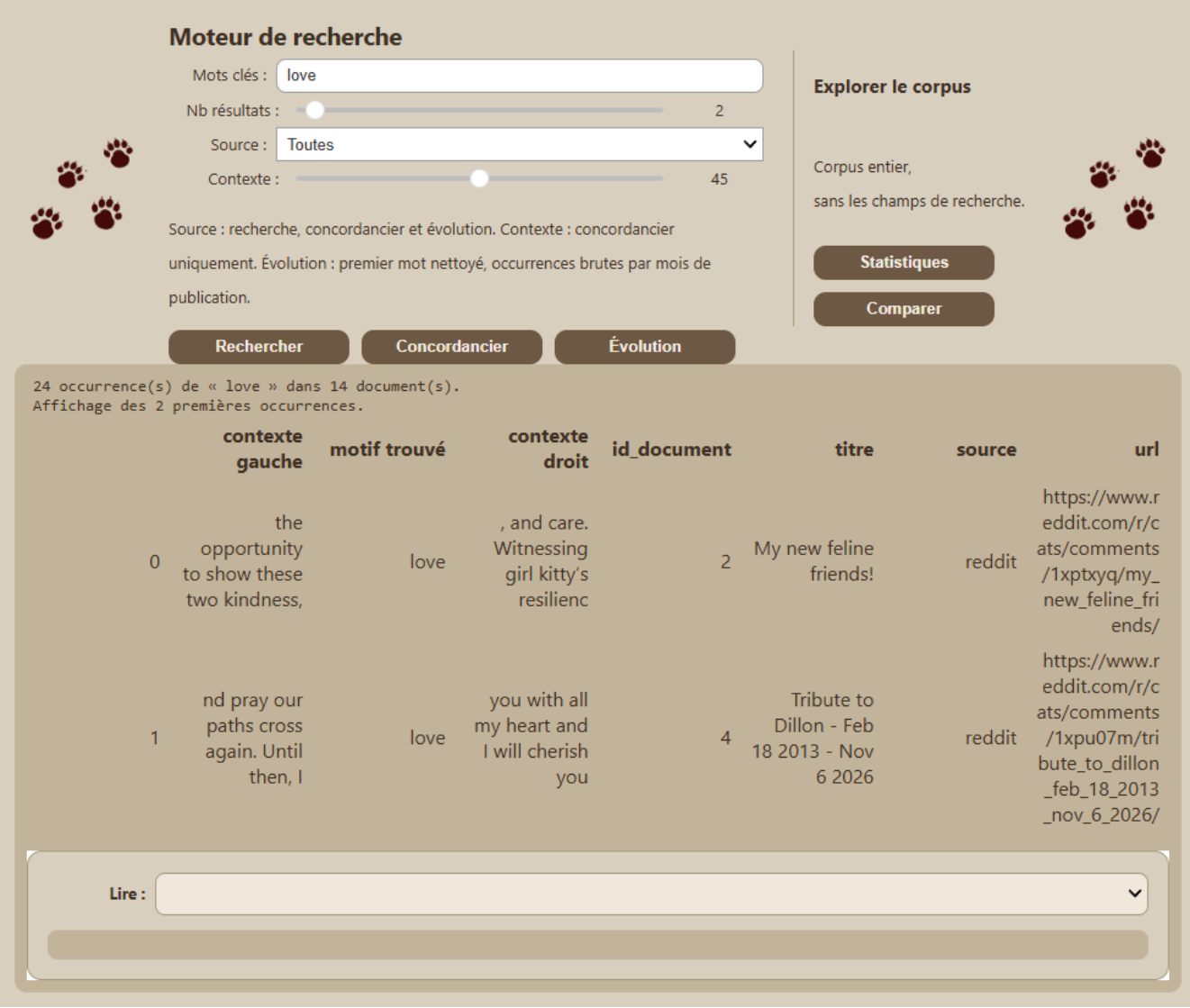
Après un clic sur Rechercher, les documents retenus apparaissent dans l'ordre décroissant des scores. Si le nombre de correspondances est inférieur au maximum demandé, le tableau comporte moins de lignes.



Sous les résultats, une liste permet de choisir un document. Le lecteur affiche alors son titre, ses informations et le texte sauvegardé.
L'utilisateur peut ensuite ouvrir le lien d'origine pour consulter la publication Reddit ou la page de l'article arXiv. Pour arXiv, le lecteur intégré ne remplace pas la consultation du texte scientifique complet qui peut être trop long.

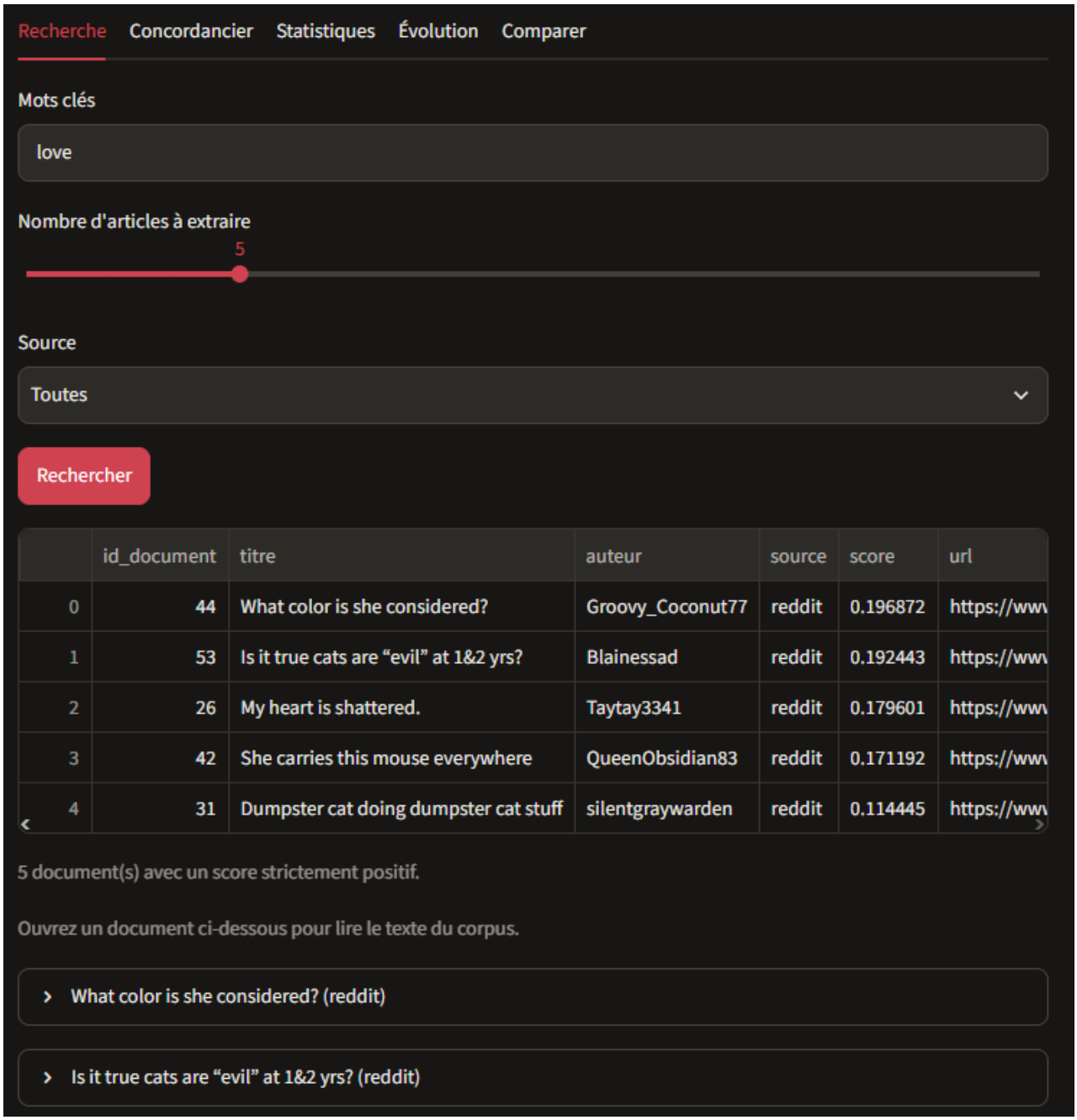


Le même mot peut être utilisé dans le concordancier. Le curseur du nombre de résultats limite alors les occurrences affichées et que le curseur de contexte règle le nombre maximal de caractères à gauche et à droite.



L'application web propose un parcours comparable. Elle se lance depuis le terminal du projet avec la commande
python -m streamlit run app.py

L'utilisateur accède ensuite aux onglets Recherche, Concordancier, Statistiques, Évolution et Comparer. Les traitements de recherche reposent sur le même moteur que dans le notebook.


Recherche
Concordancier
Statistiques
Évolution
Comparer
Mots clés
love
Nombre d'articles à extraire
5
Source
Toutes
Rechercher
id_document
titre
auteur
source
score
url
0 44 What color is she considered? Groovy_Coconut77 reddit 0.196872 https://ww
1 53 Is it true cats are “evil” at 1&2 yrs? Blainessad reddit 0.192443 https://ww
2 26 My heart is shattered. Taytay3341 reddit 0.179601 https://ww
3 42 She carries this mouse everywhere QueenObsidian83 reddit 0.171192 https://ww
4 31 Dumpster cat doing dumpster cat stuff silentgraywarden reddit 0.114445 https://ww
5 document(s) avec un score strictement positif.
Ouvrez un document ci-dessous pour lire le texte du corpus.
What color is she considered? (reddit)
Is it true cats are “evil” at 1&2 yrs? (reddit)

La validation

Tests unitaires

Une suite de tests a été ajoutée dans test_recherche.py avec la bibliothèque standard unittest. Elle vérifie des comportements de SearchEngine.search et de Corpus.concorde sur un corpus fictif de cinq documents.
Ce choix du corpus fictif rend les résultats attendus connus et évite de dépendre des API ou de modifier données réelles. Les documents sont préparés directement dans setUp, les tests ne portent pas sur la collecte, la gestion des auteurs ou la sauvegarde.
Les huit tests couvrent les comportements suivants:

Test	Vérification	Résultat attendu
Scores positifs et triés	Recherche de `cat` avec une limite de 20	Deux documents, scores positifs et ordre décroissant
Limite de résultats	Recherche avec une limite de 1	Un seul document, correspondant au premier résultat du classement complet
Requête inconnue ou vide	Recherche de `xyzabc`, puis d'une chaîne vide	Tableau vide
Source sans correspondance	Recherche de `cat` limitée à arXiv	Tableau vide
Filtre avant limitation	Recherche de `bird` sur arXiv avec une limite de 1	Un document arXiv positif, même si le meilleur résultat global est Reddit
Occurrences et documents	Concordance de `cat`	Trois occurrences dans deux documents
Filtre du concordancier	Concordance sur Reddit puis arXiv	Trois occurrences Reddit et aucune occurrence arXiv
Taille du contexte	Concordance avec un contexte de quatre caractères	Contextes de longueur inférieure ou égale à quatre

Le test des occurrences vérifie également que la casse est ignorée et que `bobcat` n'est pas compté comme le mot entier `cat`. Il illustre la différence entre le nombre de résultats documentaires et le nombre d'occurrences.
La commande de lancement est *python -m unittest -v test_recherche*



Les vérifications préparées ne démontrent pas l'absence de toute erreur. Elles ne couvrent pas toutes les méthodes du corpus, les accès aux API, le rendu des interfaces ou une valeur numérique exacte du cosinus.

Tests globaux

Les essais globaux portent sur le logiciel utilisé depuis ses interfaces. Ils vérifient les traitements, la transmission des paramètres, les messages affichés et la consultation des résultats.
Les essais progressifs ont permis de repérer plusieurs problèmes comme les résultats à score nul, les filtres incomplets et les difficultés de mise en page.

Cas testé	**Comportement attendu**	**Résultat observé**
Chargement du corpus sauvegardé	Interface utilisable sans nouvelle collecte	Pas d'appel API
Requête vide ou composée d'espaces	Message explicite, sans exception	Message de requête vide
Mot absent du vocabulaire	Aucun résultat proposé	Message de 0 résultats
Limite supérieure au nombre de correspondances	Aucun remplissage avec des scores nuls	Que les résultats au scores positives
Limite fixée à un résultat	Un seul document, le mieux classé parmi les candidats	Limite bien
Changement de source	Résultats limités à la source sélectionnée	Les résultats provenant que de la source choisie
Concordancier et contexte	Nombre d'occurrences et étendue du contexte conformes aux paramètres	Conforme
Sélection d'un document	Texte affiché correspondant au résultat choisi	Le texte bien du document choisi

Requête directe et interface	Mêmes résultats à corpus et paramètres identiques	Bien, résultats identiques
Statistiques et comparaison	Analyses globales indépendantes des mots-clés	Pas de dépendance

La maintenance

Maintenir le programme existant

La maintenance doit préserver le fonctionnement actuel, donc de conserver des dépendances cohérentes, une procédure d'installation claire et des tests relançables après chaque modification du moteur.
Les fichiers de données doivent rester documentés. Les identifiants de connexion doivent rester séparés des fichiers partagés.
Une amélioration de l'organisation serait de déplacer des traitements statistiques présents dans l'interface vers des fonctions ou modules dédiés. Cela faciliterait leur réutilisation et leur validation indépendante.

Enrichissement du corpus

L'augmentation du nombre de documents et l'ajout d'autres sources seront une évolution majeure. L'objectif serait d'améliorer la couverture du thème et la variété des informations accessibles.

Aide à la saisie

Une autre évolution serait de proposer une correction orthographique lorsqu'un mot ne figure pas dans le vocabulaire. Une distance d'édition, comme *la distance de Levenshtein* pourrait servir à rechercher des candidats proches.
La proposition devrait rester visible et contrôlable par l'utilisateur. Un remplacement automatique risquerait de modifier à tort un nom propre ou un terme scientifique.
Cette fonction nécessiterait de définir un seuil de proximité et de limiter le coût de comparaison avec l'ensemble du vocabulaire.

Autres pistes d'amélioration

- **Classement BM25**: implémenter une autre méthode de score puis comparer les résultats sur un ensemble identique de requêtes.
- **Filtres supplémentaires**: filtre sur les auteurs et les dates pour affiner la recherche.
- **Évaluation de la pertinence**: définir quelques requêtes et identifier manuellement les documents attendus pour comparer plus rigoureusement les versions du moteur.

Resume

Le projet a permis de construire une chaîne complète allant de la collecte de textes à leur consultation dans une interface. La progression en trois parties a facilité la mise en place des données, des traitements et des interactions.

Le projet a aussi apporté une expérience de l'organisation d'un programme Python. Avant, la POO était vue plus sur C# et Java.

Les difficultés liées aux noyaux Jupyter et aux environnements ont montré l'importance de la reproductibilité. Un programme doit pouvoir être relancé facilement sur plusieurs appareils différents.

La phase d'amélioration finale a été particulièrement utile. Elle a montré que la qualité d'un logiciel dépend de la cohérence des paramètres, de l'absence de résultats trompeurs et de la possibilité de consulter réellement l'information trouvée.

La partie consacrée au moteur et aux interfaces a été particulièrement intéressante sur le plan personnel. La partie de l'interface a demandé plus de recherches autonomes que la partie de récupération des données, faite pendant les heures de cours.

Le programme bien sûr reste limité à un corpus de taille modeste, à une recherche fondée sur les termes et aux textes effectivement sauvegardés. Il constitue qu'un prototype permettant d'expliquer et d'observer les mécanismes d'une recherche documentaire.

Utilisation de l'assistant et ressources

Perplexity a été utilisé comme outil d'accompagnement pour comprendre certaines étapes, proposer et corriger des portions de code, diagnostiquer des erreurs, préparer les tests unitaires et aider à la rédaction du rapport. Les propositions ont été intégrées progressivement et vérifiées par des essais. Déclaration sur honneur que le code écrit est compris personnellement.

https://github.com/AnnKomp/Projet.git